



AUTONOMOUS UNIVERSITY OF THE STATE OF MEXICO

UAEM VALLE DE MÉXICO UNIVERSITY CENTRE

**Modeling an Incremental Learning Scheme with Distributed Memory
in Artificial Neural Networks**

RESEARCH PROTOCOL

P r e s e n t e d B y

Eng. Luis Ángel González Hernández

Director: Dr. Víctor Manuel Landassuri Moreno

Co-Director: Dr. Leandro L. Minku

Tutor: Dr. Maricela Quintana Lopez

Atizapán de Zaragoza, State of Mexico. November 2025

Contents

List of Figures	2
1 Introduction	4
2 Problem Statement	5
3 Objectives	6
3.1 General Objective	6
3.2 Specific Objectives	6
4 Hypothesis	6
5 Justification	6
6 Delimitation	7
7 Consequences	7
8 Theoretical Framework	7
8.1 Optical Digit	7
8.2 Artificial Neural Networks	8
8.2.1 Activation Function	10
8.3 Multilayer Perceptron Neural Networks	12
8.4 Backpropagation Algorithm	12
8.5 Convolutional Neural Networks	13
8.6 Incremental Learning	13
8.6.1 Incremental Learning Algorithms	13
8.6.2 Memory Mechanisms in Incremental Learning	15
8.6.3 Strategies to Mitigate Catastrophic Forgetting	15
9 State of the Art	16
9.1 Search Strategy	16
9.2 Inclusion and Exclusion Criteria	17
9.3 Selection Process	17
9.3.1 Results Synthesis	18
9.3.2 Conclusion of the State of the Art	19
10 Methodology	19
11 Chapter Organisation	20
12 Gantt Diagram	20
References	21
A Phases of the Backpropagation Process	22
A.1 Forward Propagation	22
A.2 Backward Propagation	23

List of Figures

1	Optical Digit Dataset	7
2	Diagram of a basic artificial neural network.	8
3	AND logic gate in the input plane.	9
4	OR logic gate in the input plane.	9
5	XOR logic gate: example of a non-linearly separable problem.	10
6	Step Function	10
7	Sigmoid Function	10

8	ReLU Function	11
9	Softmax Function	11
10	Hyperbolic Tangent Function	12
11	Multilayer Neural Network	12
12	Two traditional approaches to incremental learning.	14
13	PRISMA 2020 flow diagram for the selection of studies.	18
14	Gantt Chart	21

Modeling an Incremental Learning Scheme with Distributed Memory in Artificial Neural Networks

November 12, 2025

Abstract

An artificial neural network is a Machine Learning model inspired by the functioning of biological neural networks. It is composed of artificial neurons (or nodes) organised into layers (input, hidden, and output), where information is processed through non-linear mathematical transformations.

The learning process of these models is carried out through training, during which data pass through the network layer by layer. In each neuron, an activation function is computed, the loss is estimated by comparing the output with the actual value, and then the backpropagation algorithm is applied to adjust the weights and biases in order to minimise this loss. This process is repeated until the loss converges.

In the traditional approach, the model is trained with an initial dataset and, upon the arrival of new information, it must be retrained from scratch using all available data. This process can be inefficient and computationally expensive, especially with large data volumes.

Incremental learning, a branch of Artificial Intelligence, allows updating the model's knowledge without the need to retrain it completely with historical data [2]. In this work, Artificial Neural Networks focused on handwritten digit classification will be used, employing the backpropagation algorithm in Multilayer Perceptron networks. A weight duplication strategy will be explored to simulate short- and long-term memory mechanisms, aiming to improve upon the results presented in [2].

Keywords: Incremental Learning, Artificial Neural Networks, Digit Classification.

1 Introduction

Artificial Intelligence (AI) is a field of knowledge that seeks to develop machines capable of emulating human behaviour and reasoning, enabling interaction almost indistinguishable from that with another human being. This area not only focuses on the creation of intelligent systems but also on developing tools that facilitate and optimise everyday activities.

A subfield of AI is Machine Learning, which studies algorithms capable of autonomously learning tasks. Within this field, Artificial Neural Networks (ANNs) are among the most recognised techniques, as they employ mathematical processes to solve complex problems. ANNs have proven useful in areas such as predicting 5G network capacities based on daily traffic [24], as well as in the classification of materials such as metals and rocks using fuzzy logic [21].

ANNs are composed of artificial neurons that simulate biological ones. In this context, the chemical processes that occur in the brain are computationally replicated through signals that travel between artificial neurons, hereafter referred to as "neurons". This distributed and parallel structure grants ANNs a remarkable learning capability [14].

In the context of machine learning, when a technique such as ANNs focuses on learning a specific task, it is considered a memoryless linear algorithm. Since its origins, this has been one of the most widely used approaches in ANNs [7]. However, when it becomes necessary to incorporate new information without retraining the entire model, the concept of Incremental Learning arises, which seeks to include new data without having to train the model from scratch.

A related aspect derived from incremental learning is the concept of memory in AI, which explores how a machine learning algorithm can forget previously acquired information when incorporating new data. This problem, analogous to human memory loss, poses challenges for both machines and human beings. In response,

several approaches have been designed to mitigate this phenomenon. One example is the work of [2], which proposes the use of ANNs with dual weights, where the first layer of weights behaves as short-term memory and the second as long-term memory. The experiments conducted in [2] demonstrate improvements in incremental learning tasks, reducing information loss compared with previous implementations such as the Learn++ algorithm [12, 5].

This research aims to explore new configurations of duplicated weights, expanding and extending the results obtained in [2].

2 Problem Statement

Artificial Neural Networks (ANNs), especially Multilayer Perceptrons (MLPs), have the ability to learn tasks and solve prediction and classification problems. Using learning algorithms such as Backpropagation, it is possible to adjust the weights of an ANN so that it can address a specific task. MLPs are a common form of ANN, composed of multiple layers of neurons, where the input layer receives the data and the output layer provides the predictions. The intermediate layers, also known as hidden layers, perform non-linear transformations that allow the network to learn complex representations of the data. However, many of the tasks solved in everyday life generate additional information over time. An example of this is the behaviour of a financial time series or weather prediction for a given region. In this context, incremental learning arises as a scarcely explored approach that allows the model to learn new information without the need to retrain the entire model with both old and new data.

In current machine learning models, when a dataset is used to train a specific model, that model is functional only for that dataset and the information it represents. However, when it becomes necessary to incorporate new information into the model, that new data must be collected, added to the existing dataset, and then the entire model must be retrained to integrate the new data.

In this context, if a network trained with an initial dataset d_1 must later be trained with a second dataset d_2 , the knowledge acquired from d_1 will be lost. If an incremental learning model is not used, d_1 and d_2 must be combined into a single dataset and the ANN retrained to incorporate the new knowledge (d_2). On the other hand, if incremental learning is applied, the ANN is first trained with d_1 and then with d_2 , maintaining minimal information loss from d_1 . If additional information about the problem (d_3) is obtained in the future, the ANN will be trained with d_3 using the incremental model, minimising the loss of data from d_1 and d_2 .

This work is based on the research of [2], which uses a configuration of duplicated weights in the ANN. In this configuration, the ANN's weights are duplicated, and different learning rates are assigned to the layers. The first layer, associated with a high learning rate, simulates short-term memory by learning a new task quickly. This research is based on the study of [2], which proposes a strategy to improve the incremental learning capability of Artificial Neural Networks (ANNs) through weight duplication. In this approach, each neural connection has two versions of its weights: one that simulates short-term memory and another that represents long-term memory.

Short-term memory uses a high learning rate, allowing it to adapt quickly to new information but also making it more prone to forgetting previous knowledge. In contrast, long-term memory is trained with a low learning rate, learning more slowly but retaining previously acquired information more effectively.

During training, both sets of weights are updated in parallel according to their respective learning rates. When making a prediction, the results are obtained from a combination of both sets of weights. This strategy allows the neural network to integrate both recent and accumulated knowledge, emulating the complementary functioning of biological memory systems and forgetting previous information more gradually. The second layer, with a low learning rate, simulates long-term memory, learning more slowly and forgetting less prior information. By integrating both layers, the ANN balances the output to consider both the newly acquired information and the information retained in both weight layers.

The main problem of incremental learning in [2] is that, as new datasets are incorporated, the knowledge acquired from the earlier datasets is progressively lost, which becomes less useful if, in the future, there are 10 or 20 incremental training stages with new data.

Therefore, it is crucial to explore new ANN configurations that improve current methods to reduce the amount of information forgotten as new data arrive. As in previous research, this work will be based on the concepts of short- and long-term memory. However, instead of duplicating the weights and having two learning rates, the idea of creating more copies of the weights will be proposed, each with a distinct learning rate.

3 Objectives

3.1 General Objective

To design an artificial neural network for incremental learning based on the principle of short- and long-term memory, aiming to use more than two layers of duplicated weights for digit recognition, achieving a lower information loss compared with previous works.

3.2 Specific Objectives

1. Implement the algorithm presented in [2] for digit recognition through short- and long-term learning, using the parameters indicated therein.
2. Obtain the Optical Digits dataset, clean the data, and prepare it according to the procedure described in [2].
3. Split the dataset into training and testing subsets as explained in [2], and test the first implemented code to verify the results previously reported in [2].
4. Use the implemented algorithm as a basis and extend it to allow more than two duplicated weights, applying the previously mentioned dataset.
5. Compare both implementations in search of a significant reduction in learning rates with respect to previous works reported in the literature.

4 Hypothesis

The inclusion of more than two layers of duplicated weights, each with their respective learning rates, can reduce the forgetting of previously learned information in an incremental learning model using Multilayer Perceptron (MLP) artificial neural networks when the Backpropagation algorithm is applied.

5 Justification

Artificial Neural Networks (ANNs) enable machine learning and the resolution of a wide variety of complex problems. However, as mentioned previously, traditional machine learning techniques present a significant limitation: when new data blocks are incorporated, the model tends to degrade its performance and forget previously learned information [2].

Although current results in the field have shown significant progress, neural networks still possess short-term memory that deteriorates over time. In contrast, human beings are capable of learning new tasks without significantly forgetting what they previously learned. While this ability is supported by the biological structure of the brain, in the computational domain it is possible to explore configurations that partially emulate such behaviour, improving the retention of acquired knowledge.

From a technical point of view, there are no restrictions preventing experimentation with new architectures or training strategies that allow ANNs to accumulate information progressively without suffering from overwriting or loss of prior knowledge. This opens the possibility of developing more efficient and sustainable models applicable in various contexts of modern artificial intelligence.

In a scenario where incremental learning is not employed, the incorporation of new data would require retraining the entire model with all available information, which is a costly process in terms of time, energy, and computational resources. In contrast, an incremental approach allows the model to be trained only with new data, significantly reducing computational and energy costs, and making its application in real environments more feasible.

At present, various tools facilitate the implementation of artificial neural networks through specialised libraries. In this research, the development environment Google Colab will be primarily used, as it provides a free infrastructure based on GPU and full compatibility with the Python language, one of the most widely used in data science and artificial intelligence.

For the model implementation, the PyTorch library will be used, as it is one of the most powerful and flexible frameworks for deep learning development. PyTorch stands out for its dynamic nature, which allows networks to be defined and modified flexibly during execution, in addition to offering clear syntax that closely resembles pure Python. These characteristics facilitate rapid experimentation, error debugging, and integration

with other scientific packages. Therefore, PyTorch represents an ideal tool for the construction and evaluation of incremental learning models in this research.

6 Delimitation

In this research, only Multilayer Perceptron (MLP) neural networks will be used. It should be noted that this is not the only type of neural network that exists, as there are also Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Radial Basis Function Networks (RBFNs) [20]. Furthermore, optimisation techniques such as genetic algorithms will not be addressed, limiting the study exclusively to exploring performance improvements in neural networks with more than two duplicated-weight layers within an incremental learning context. Likewise, the study is restricted to the aforementioned networks, excluding deep neural models and the use of other datasets.

7 Consequences

If the experiment is successful, the following consequences are expected:

1. The forgetting of previously learned information will be reduced.
2. The learning time will be shorter.

As mentioned previously, it will be possible to incorporate new data without the need to retrain the model with all the existing information.

Classification or prediction tasks will be able to integrate new information into artificial neural networks without having to retrain the model with all historical data, thus reducing the bottleneck associated with the training process.

8 Theoretical Framework

8.1 Optical Digit

The *Optical Digit* dataset is a classical benchmark in pattern recognition and classification algorithm evaluation, especially in the context of *OCR* (Optical Character Recognition). Its original purpose was to provide a simple, reproducible, and balanced testbed for comparing supervised learning models without the computational cost of high-resolution images. In other words, it is a lightweight yet expressive *benchmark* for studying the generalisation capacity of different approaches.

It consists of 8×8-pixel greyscale images (as shown in Figure 1); each image represents a digit in the range 0–9. The dataset contains a total of 5,620 images and maintains a class balance: each digit has the same number of samples. This is a relevant feature, as avoiding class-frequency bias allows metrics (accuracy, F1, etc.) to more fairly reflect differences between models and preprocessing *pipelines*.

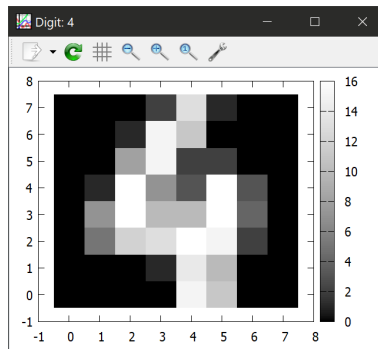


Figure 1: Optical Digit Dataset

Thanks to its balance between simplicity and usefulness, *Optical Digit* is employed in various areas of artificial intelligence, such as:

1. Digit classification [15].
2. Pattern recognition [9].
3. Model comparison and benchmarking [16].
4. Training and teaching in supervised learning [10].

Its advantages include: (i) very fast training and validation curves (ideal for iterating hyperparameters and architectures), (ii) ease of experiment reproduction (minimal transformations and preprocessing), and (iii) usefulness as a *case study* for methodologies such as incremental learning, where it is relevant to observe the effect of introducing sequential data blocks without excessive computational cost.

8.2 Artificial Neural Networks

Artificial Neural Networks (ANNs) are computational models inspired by the functioning of the human brain, designed to process information in a parallel and distributed manner. Each network consists of a set of simple processing units called *artificial neurons*, interconnected through weighted links known as *synaptic weights*. These weights represent the strength of the connection between neurons and determine the influence that one input exerts over another within the network. In this sense, the learning of an ANN essentially consists of the progressive adjustment of these weights to minimise the error between the generated and expected outputs.

Analogous to neurophysiology, where synaptic connections are strengthened or weakened based on experience, ANNs learn by modifying the numerical weights of their connections through training algorithms. This adaptive capability gives neural networks the ability to recognise patterns, classify information, make predictions, and approximate complex functions without explicit programming.

The simplest neuron model—known as the *perceptron*—is the fundamental unit of ANNs. Introduced by Rosenblatt in 1958, the perceptron aims to emulate the behaviour of a biological neuron by performing a weighted combination of its inputs, followed by an activation function that determines its output. Mathematically, this model can solve *linearly separable* problems, in which data can be divided by a linear boundary in the input space.

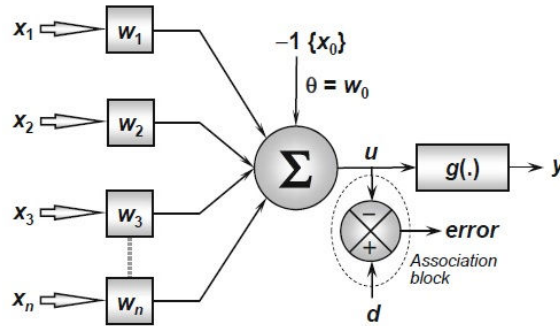


Figure 2: Diagram of a basic artificial neural network.

In Figure 2, the symbol Σ represents the summation operator. Let $x_1, x_2, \dots, x_n$ be the input variables and $w_1, w_2, \dots, w_n$ the weights associated with each synaptic connection. The neuron first computes a linear combination of the inputs weighted by their respective weights and adds an additional term known as the *bias* (b), which allows shifting the decision boundary with respect to the origin. The mathematical model of the neuron can be expressed as:

$$y = \sum_{i=1}^n w_i x_i + b,$$

where y represents the output of the neuron. The bias plays an essential role as it introduces flexibility to the model, allowing the adjustment of the decision boundary position without altering the weight values. Thus, a single neuron acts as a linear classifier, separating the input data through a hyperplane in the feature space.

However, the limitations of the perceptron become evident when dealing with non-linear problems, where a single boundary is insufficient for correctly separating the classes. This restriction led to the development of

architectures with multiple layers of neurons and non-linear activation functions, known as *multilayer neural networks* (MLPs), which allow modelling more complex relationships between input and output variables.

A single neuron can solve tasks such as AND or OR, but not non-linear problems (for example, XOR). In these cases, architectures with multiple layers are required, supported by activation functions that introduce the necessary non-linearity to model complex decision surfaces.

Logic gates and linear separability in ANNs. Binary logic gates illustrate the relationship between linear separability and the representational capacity of an artificial neural network.

- AND: produces 1 only when both inputs are 1. It is linearly separable; a perceptron can implement it, for example, with $w_1 = w_2 = 1$ and $b = -1.5$. The boundary $x_1 + x_2 - 1.5 = 0$ correctly separates the only positive case $(1, 1)$.

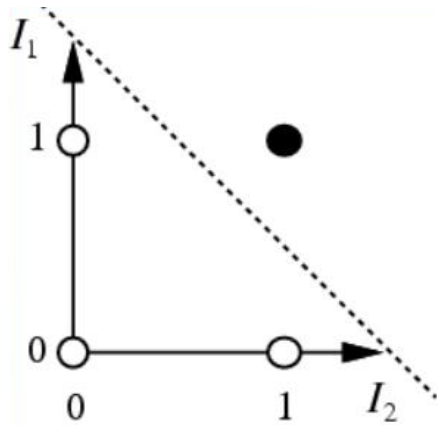


Figure 3: AND logic gate in the input plane.

- OR: produces 1 when at least one input is 1. It is also linearly separable; a perceptron can implement it with $w_1 = w_2 = 1$ and $b = -0.5$. The boundary $x_1 + x_2 - 0.5 = 0$ separates the three positive cases $(1, 0)$, $(0, 1)$, $(1, 1)$ from the negative one $(0, 0)$.

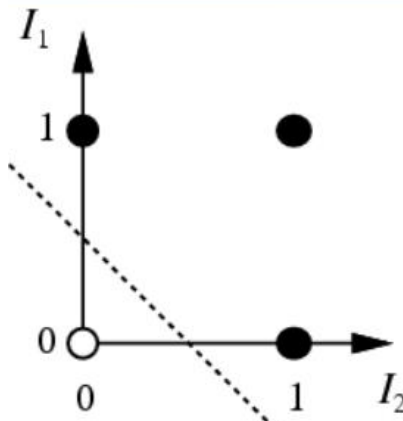


Figure 4: OR logic gate in the input plane.

- XOR: produces 1 when exactly one of the inputs is 1. It is not linearly separable, as no line can simultaneously divide the positive cases $(1, 0)$, $(0, 1)$ from the negative ones $(0, 0)$, $(1, 1)$. To solve XOR, at least one hidden layer (MLP) is required, combining multiple linear boundaries into a non-linear decision surface.

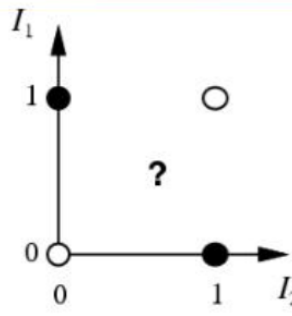


Figure 5: XOR logic gate: example of a non-linearly separable problem.

This relationship is fundamental in neural architecture design: when data are not separable by a hyperplane, a multilayer network with suitable activation functions can compose linear boundaries into non-linear shapes, thus correctly representing the problem (see Section 8.3).

8.2.1 Activation Function

Activation functions are the component that transforms a stacked set of linear transformations into a non-linear model. Without them, even multiple layers would be equivalent to a single linear transformation. In problems where the input–output relationship is inherently non-linear (as in most real-world phenomena), activation functions enable the curvature of the decision boundary and stabilise gradient flow during training [18].

Common activation functions include:

- **Step Function:**

Suitable for simple binary decisions (0/1). Its modern use is educational, as its non-differentiability complicates gradient-based training:

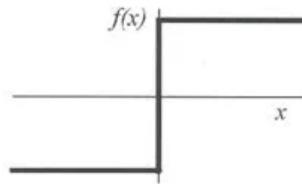


Figure 6: Step Function

$$f(x) = \begin{cases} 0 & : x < 0 \\ 1 & : x \geq 0 \end{cases}$$

- **Sigmoid Function:**

Produces outputs in $[0, 1]$ and allows a probabilistic interpretation (e.g., $0.8 \Rightarrow 80\%$). It also smooths decisions and facilitates gradient computation:

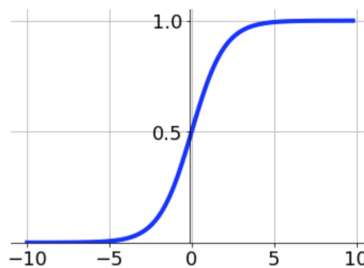


Figure 7: Sigmoid Function

$$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}.$$

Its parameters are adjusted through backpropagation (see Section 8.4).

- **Rectified Linear Unit (ReLU):**

The de facto standard in deep networks due to its efficiency and ability to mitigate the *vanishing gradient* problem. It activates positive values proportionally and nullifies negative ones, accelerating convergence:

- *Vanishing gradient*: phenomenon where gradients become very small and early layers stop updating effectively.

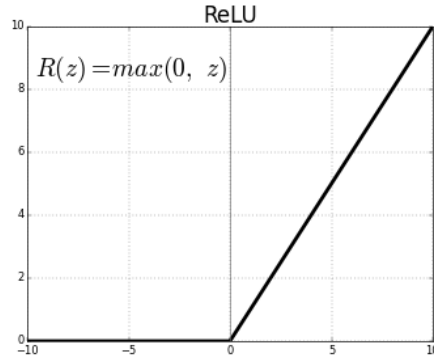


Figure 8: ReLU Function

$$f(x) = \begin{cases} 0 & : x < 0 \\ x & : x \geq 0 \end{cases}$$

- **Softmax Function:**

Used in the output layer for multi-class classification, converting activations into probabilities that sum to 1:

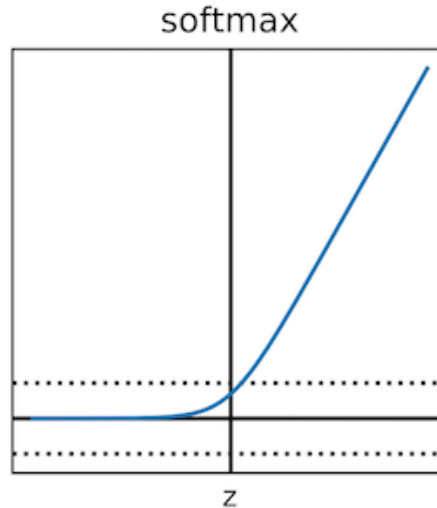


Figure 9: Softmax Function

$$f(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}.$$

- **Hyperbolic Tangent Function:**

Similar to the sigmoid function but with range $[-1, 1]$, useful for centring data around zero; in very deep networks, it may still suffer from vanishing gradients:

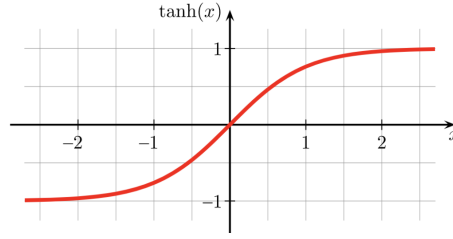


Figure 10: Hyperbolic Tangent Function

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

In summary, ANNs enable tackling non-linear relationships, learning complex input–output mappings, and obtaining models that are more robust to noise. Among the most widespread architectures are multilayer perceptrons and convolutional networks, described below.

8.3 Multilayer Perceptron Neural Networks

Multilayer Perceptron (MLP) Neural Networks consist of an input layer, one or more hidden layers, and an output layer. The hidden layers contain neurons with activation functions (see Section 8.2.1), responsible for breaking linearity and building decision surfaces in higher-dimensional spaces. Thus, problems that a single neuron cannot solve (such as XOR) become tractable by combining multiple layers and activations.

Figure 11 illustrates a 4-layer MLP: one input, two hidden, and one output. The input layer injects the variables; non-linear processing occurs in the hidden layers (and, in certain designs, also in the output). This modular structure facilitates adding depth or width according to the problem’s complexity.

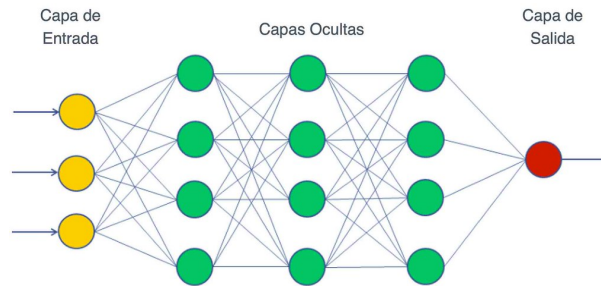


Figure 11: Multilayer Neural Network

8.4 Backpropagation Algorithm

The backpropagation algorithm allows the adjustment of the network parameters (weights and biases) based on the gradient of a loss function. It was key to extending the capability of the simple perceptron to deep networks capable of solving non-linear problems.

Operationally, the network first performs a forward propagation to compute the output and the error; the error is then propagated backwards (backpropagation) by applying the chain rule to obtain the partial derivatives with respect to each parameter. With these derivatives, weights and biases are updated through an optimiser (gradient descent or others). The process requires differentiable activation functions and an appropriate loss function [19].

The flow is summarised in two phases:

1. **Feed-forward:** Propagation of inputs forwards to obtain outputs and identify the error.
2. **Backpropagation:** Backward propagation of the error and parameter updates layer by layer.

For a detailed description, see Appendix A (Sections A.1 to A.2).

8.5 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are deep architectures specialised in data with spatial structure (images, visual sequences, local patterns). Their success relies on three complementary types of layers:

- **Convolutional:** apply local filters that detect edges, textures, and shapes; they allow partial translational invariance.
- **Pooling:** reduce dimensionality and computational cost while preserving salient features.
- **Fully connected:** integrate the extracted information and perform classification.

This design has proven effective in computer vision, medical image diagnosis, robotics, and speech recognition, among other fields.

8.6 Incremental Learning

The constant increase in the amount of available data and the demand for systems capable of learning continuously have driven the development of Incremental Learning. In this approach, training examples are presented sequentially, and the model must adapt to new information without forgetting previously acquired knowledge. Unlike traditional supervised learning schemes—in which the entire dataset is processed simultaneously—incremental learning trains the model progressively, gradually integrating new patterns or tasks [7].

This feature is essential in dynamic or non-stationary environments, where data change over time and it is not feasible to carry out full retraining at each update. In fields such as robotics, systems must constantly adapt to new scenarios, lighting conditions, textures, or human behaviours without forgetting prior information. Similarly, in applications such as cybersecurity, image processing, speech recognition, or recommender systems, the continuous data flow requires models capable of adjusting their knowledge in real time without degrading historical performance.

Incremental learning is directly inspired by the way human beings learn: people are able to incorporate new knowledge over time while retaining relevant information from past experiences. For neural networks, replicating this type of learning involves balancing two key properties: plasticity, which allows adaptation to new data, and stability, which preserves previously learned knowledge. This balance, known as the stability–plasticity dilemma, is one of the main challenges of incremental learning.

However, conventional neural networks exhibit a phenomenon called catastrophic forgetting, in which the incorporation of new tasks or classes leads to the loss of previously learned information. This occurs because the optimisation process adjusts the network’s weights to adapt to the new data, overwriting previous knowledge. As a result, the model “forgets” past tasks when faced with new training sets [14].

To mitigate this problem, various strategies have been proposed within incremental learning, including:

- **Regularisation-based methods:** impose constraints on the weights most important for previous tasks, preventing them from changing drastically during the learning of new tasks. Examples include Elastic Weight Consolidation (EWC) and Synaptic Intelligence (SI).
- **Replay methods:** store a small number of representative examples or generate synthetic data to recall prior knowledge while learning new tasks.
- **Parameter isolation methods:** reserve subsets of neurons or specific layers for each task, preventing direct interference between them.

The general objective of these techniques is to maintain the ability to learn continuously without retraining the model from scratch or permanently storing previous data. In this way, incremental learning is consolidated as a central paradigm of Continual Learning, enabling the development of more efficient, adaptive, and autonomous intelligent systems. In short, it seeks to bring artificial neural networks closer to human cognitive behaviour, where knowledge is built progressively and preserved over time.

8.6.1 Incremental Learning Algorithms

In general terms, an incremental learning algorithm seeks to maintain the ability to learn continuously without retraining the entire model from scratch. These algorithms are designed to operate under the premise that data are not presented statically or simultaneously, but as a sequence of experiences or information blocks that must

be processed as they arrive. The central challenge is to update the model parameters without compromising previously acquired knowledge, avoiding catastrophic forgetting.

To fulfil this role, an incremental algorithm must meet the following essential criteria [1]:

1. Learn and update with each new datum, labelled or unlabelled, without requiring a global retraining process.
2. Preserve previously acquired knowledge, integrating new information without degrading prior learning.
3. Avoid dependence on permanent access to the original data (learning without full storage of past examples).
4. Detect and create new classes or clusters when the data introduce previously unseen patterns, as well as split or merge existing clusters according to the environment.
5. Maintain a dynamic and adaptive nature in the face of conceptual or distributional changes in the data (concept drift).

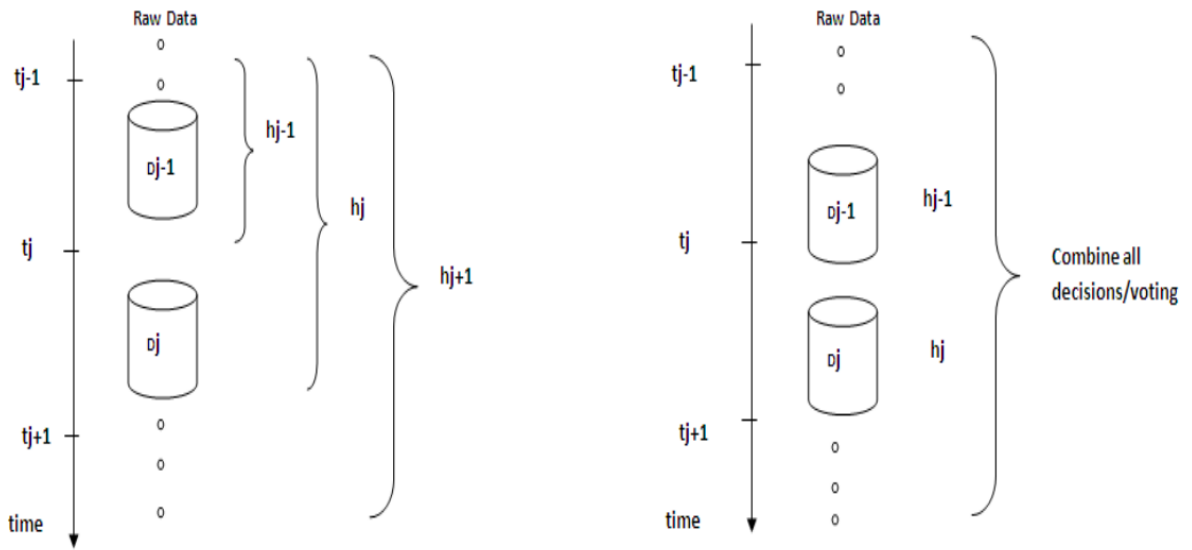


Figure 12: Two traditional approaches to incremental learning.

As shown in Figure 12, there are two fundamental historical approaches to incremental learning:

- **Cumulative or sequential approach:** each time a new data block D_j arrives, the model discards the previous hypothesis h_{j-1} and trains a new hypothesis h_j using all available data. Although global accuracy improves, this method is computationally expensive and does not represent truly continuous learning.
- **Purely incremental approach:** the model receives D_j and directly updates the parameters of the existing hypothesis h_{j-1} , relying solely on the new information. In some cases, voting mechanisms or hypothesis combinations are used to integrate changes stably between successive iterations.

In both cases, the main practical advantage of the incremental approach lies in the fact that the acquired knowledge is not stored as individual examples, but is encoded in the model parameters (weights, biases, or probabilities), thereby avoiding the storage of large volumes of historical data.

From a formal point of view, given a sequence of training examples $e_1, e_2, \dots, e_s$, an incremental algorithm generates a series of hypotheses $h_0, h_1, \dots, h_n$ such that each hypothesis h_{i+1} depends only on the previous hypothesis h_i and the current example e [7]. This principle enables adaptive and continuous learning, where the model “evolves” with each interaction.

A classic example of incremental learning is the COBWEB model [6], which performs hierarchical probabilistic categorisation. COBWEB analyses each incoming instance and decides whether to create a new category, update an existing one, or reorganise the hierarchy, based on a global utility measure. In this way, the

system builds a knowledge tree that is updated as new data are received, without the need to restart the learning process.

As the field has advanced, various families of incremental algorithms have been proposed, classified according to their main strategy:

- **Regularisation methods:** restrict changes to parameters most relevant to previous tasks, as in Elastic Weight Consolidation (EWC), Synaptic Intelligence (SI), or Memory Aware Synapses (MAS).
- **Replay methods:** employ a small memory of examples or generative models to recall previous tasks during the training of new ones. Examples include iCaRL and Generative Replay.
- **Parameter isolation methods:** assign subsets of neurons or specific layers to different tasks, reducing interference between knowledge (Progressive Neural Networks, PackNet).
- **Hybrid methods:** combine regularisation, replay, and dynamic expansion to balance stability and plasticity in non-stationary contexts.

Within this line of research, the work of [2] represents an important reference point, as it introduces an alternative approach based on dual memory: a short-term memory responsible for storing and consolidating recent information, and a long-term memory that preserves already stabilised knowledge. This mechanism is implemented through a duplication of synaptic weights (two copies of the same set of connections) that allows differentiated control of forgetting rate and learning speed.

The model proposed by Bullinaria employs incremental learning in which each data block updates only the short-term network; then, the consolidated information is gradually transferred to the long-term network, thus avoiding the direct overwriting of the main weights. This structure emulates, in a simplified manner, the memory consolidation processes observed in the human brain (hippocampus–neocortex), offering an efficient alternative to mitigate catastrophic forgetting without resorting to explicit storage of previous data.

In the present work, this principle is adopted and extended, exploring the possibility of using more than two sets of duplicated weights to analyse whether a higher degree of partitioning in synaptic memory can offer better stability and knowledge retention throughout the incremental learning process.

8.6.2 Memory Mechanisms in Incremental Learning

The human learning process combines multiple levels of memory that interact hierarchically. Inspired by this principle, incremental learning in neural networks seeks to emulate the distinction between short-term memory (recent and volatile information) and long-term memory (consolidated and stable knowledge).

In the computational context, these mechanisms are represented by different storage structures or duplication of synaptic weights. Models such as that proposed by [2] use two sets of weights: one for rapid updating (similar to the hippocampus) and another for slow consolidation (analogous to the neocortex). This architecture allows plasticity to be maintained during the learning of new patterns without compromising the stability of previously acquired knowledge.

The study of these interactions forms the basis for the design of the present work, which proposes extending the dual-memory approach towards a scheme of multiple retention levels, in order to evaluate its impact on information loss and the efficiency of incremental learning.

8.6.3 Strategies to Mitigate Catastrophic Forgetting

One of the greatest challenges in incremental learning is catastrophic forgetting, a phenomenon that occurs when a neural network loses part or all of the knowledge previously acquired when learning new tasks or incorporating recent data. This problem arises because the optimisation process adjusts the network’s weights to fit the new information, overwriting parameters that previously represented prior knowledge. Consequently, the model may show excellent performance on the most recent task but poor performance on earlier ones.

To mitigate this problem, the literature proposes different approaches that seek to balance two fundamental principles: plasticity (ability to adapt to new tasks) and stability (ability to retain previous knowledge), commonly referred to as the stability–plasticity dilemma. The main strategies used to counteract catastrophic forgetting in incremental learning are described below:

1. Regularisation-based methods:

These methods add constraints to the weight update process, preventing those that proved important for previous tasks from changing drastically during the learning of new ones. A classic example is Elastic Weight Consolidation (EWC), which computes a parameter-importance matrix and penalises

modifications to them [11]. Modern variants such as Synaptic Intelligence (SI) or Memory Aware Synapses (MAS) follow a similar principle: they maintain a “synaptic memory” of the parameters most relevant to previous tasks, reinforcing their preservation.

2. Replay methods:

These approaches preserve a small memory with representative examples from the past or use generative models to recreate synthetic data. During the training of new tasks, the network periodically revisits this memory to keep previous information active. Examples of such techniques include Incremental Classifier and Representation Learning (iCaRL) and Generative Replay based on Generative Adversarial Networks (GANs) or Variational Autoencoders (VAEs).

3. Parameter isolation or partitioning methods:

Instead of reusing the same weights for all tasks, these methods assign subsets of neurons or specific layers to each new task, reducing interference between them. Models such as Progressive Neural Networks or PackNet gradually expand the architecture, preserving learned connections and adding new capacity without modifying existing ones. This approach is useful in contexts where the number of tasks is limited or known in advance.

4. Distillation and knowledge transfer methods:

Inspired by Knowledge Distillation, these models transfer information from a trained network (“teacher”) to a new network (“student”), seeking to preserve learned internal representations. During incremental learning, the student network learns the new tasks while attempting to replicate the teacher’s predictions for previous tasks, thus reducing the degree of forgetting.

5. Hybrid or multi-strategy methods:

In the most recent systems, the above strategies are combined to achieve a more efficient balance between stability and plasticity. These models integrate regularisation, selective replay, and dynamic architectural expansion, adapting their behaviour according to the nature of the data or tasks. Examples include Dynamic Architecture Networks or Meta-Incremental Learning schemes, where the system learns to adjust its own consolidation mechanism.

Each of these approaches seeks, to some extent, to emulate human cognitive processes of learning and retention. However, most require manual control of model complexity or partial storage of old data, which limits their scalability in real-world environments.

In this context, the model proposed by [2] represents a biologically inspired alternative that avoids the use of explicit memories or external structures, instead implementing a dual-memory system through duplication of synaptic weights. In this way, new knowledge is gradually consolidated, reproducing the process of information transfer from the hippocampus to the neocortex in the human brain.

The present work takes up this idea and proposes its extension, assessing whether the use of multiple weight duplications and different levels of consolidation can offer a more robust and scalable solution to catastrophic forgetting in incremental learning.

9 State of the Art

9.1 Search Strategy

For the development of this state of the art, the PRISMA 2020 methodology (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) was adopted, which provides a standardised framework to conduct systematic reviews of scientific literature in a transparent, rigorous, and reproducible manner.

The objective was to identify and analyse relevant research related to incremental learning, continual learning, and strategies to mitigate *catastrophic forgetting* in artificial neural networks.

The bibliographic search was carried out from **2010 to 2025**, with the aim of covering both foundational works—such as those proposed by Bullinaria and other authors in the 2010s—and the most recent advances in the field. The period was selected considering that, from 2010 onwards, scientific interest in retention and forgetting mechanisms within machine learning began to consolidate.

The search process was conducted between 2010 and 2025 in the following academic databases:

- IEEE Xplore
- ScienceDirect

- SpringerLink
- Scopus
- Google Scholar
- Advancing Computing as a Science & Profession
- arXiv (Cornell University)

Boolean combinations of keywords were used in both English and Spanish, including: “*incremental learning*”, “*lifelong learning*”, “*continual learning*”, “*catastrophic forgetting*”, “*artificial neural networks*” and “*artificial intelligence*”. The terms were adapted to each database’s search options in order to maximize coverage and retrieve the most relevant and up-to-date studies.

9.2 Inclusion and Exclusion Criteria

With the purpose of ensuring the relevance and methodological quality of the selected studies, the following criteria were established:

Inclusion criteria:

- Publications between 2010 and 2025.
- Articles that explicitly address incremental or continual learning in artificial neural networks.
- Research that analyses the phenomenon of *catastrophic forgetting* or proposes associated solutions.
- Studies with applications in areas such as robotics, computer vision, cybersecurity, or adaptive systems.

Exclusion criteria:

- Review articles without empirical evidence.
- Works without full-text access or with preliminary results.
- Duplicate publications or those with evident methodological bias.

9.3 Selection Process

The initial search yielded a total of 1,246 records. After removing 312 duplicates, 934 unique articles were retained. During the screening phase (title and abstract review), 886 studies were discarded for not meeting the inclusion criteria. The remaining 48 articles were evaluated in full text, and finally, 18 met the required methodological and thematic standards for qualitative synthesis.

The selection process is summarised in Table 1 and in the PRISMA flow diagram shown in Figure 13.

Table 1: Summary of the article selection process according to PRISMA 2020.

Stage	Number of studies (n)
Records identified in databases	60
Duplicates removed	15
Records after duplicate removal	45
Records excluded after title/abstract screening	30
Articles assessed in full text	28
Articles excluded for lack of applicable results	2
Studies included in the qualitative synthesis	18

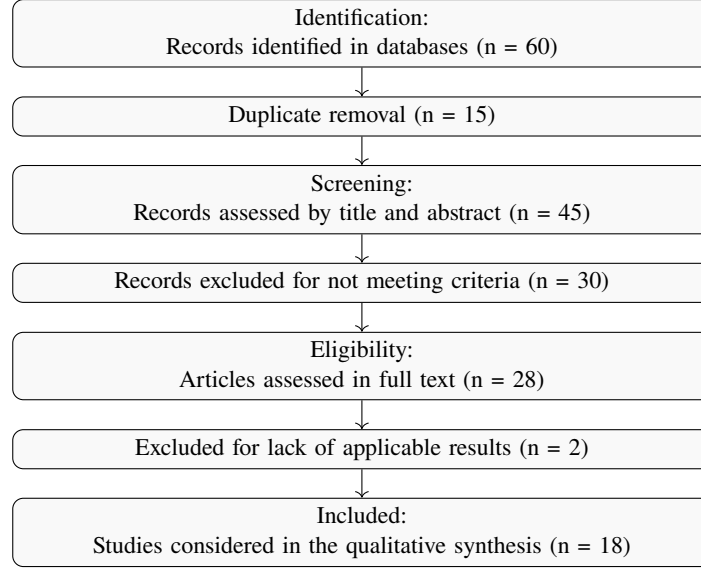


Figure 13: PRISMA 2020 flow diagram for the selection of studies.

9.3.1 Results Synthesis

The 18 studies analysed provide a comprehensive overview of incremental learning in artificial neural networks, addressing different domains, architectures, and strategies to mitigate catastrophic forgetting. The comparative analysis enabled the identification of six main thematic lines, reflecting both conceptual evolution and the most relevant practical applications in the area:

- **Robotics and adaptive control:** In this field, incremental learning is mainly applied to the control of intelligent prostheses, exoskeletons, and autonomous robots, where systems must continuously adapt to changing environmental or user conditions. Lippi et al. [13] highlight that incremental models enable dynamic recalibration without the need for global retraining, favouring more precise and energy-efficient responses. These findings support the idea of maintaining dual-memory mechanisms, where one part of the system consolidates stable experiences while another adjusts to new inputs.
- **Computer vision and pattern recognition:** In this line, there is extensive use of *CNNs* with continual learning mechanisms applied to **image classification, semantic segmentation, and autonomous driving**. Wei et al. [23] show that combining *replay* strategies with selective regularisation improves the retention of visual features. However, these methods often depend on storing previous samples, which entails higher computational and memory costs. In this context, biologically inspired models such as [2] offer a lighter alternative by not requiring explicit memory.
- **Cybersecurity and traffic analysis:** Cerasuolo et al. [3] implement the *MEMENTO* model for incremental classification of encrypted traffic, demonstrating the applicability of continual learning in anomaly detection scenarios. Their approach combines weight consolidation strategies with temporal stability metrics, reinforcing the relevance of hierarchical memory management in systems that must respond in real time.
- **General methods and theoretical approaches:** Among the most influential advances are the elastic regularisation methods proposed by Kirkpatrick et al. [11], episodic memory mechanisms such as *iCaRL* [17], and architectures with progressive neuronal expansion described by [2]. These proposals reflect different approaches to the stability–plasticity dilemma: while EWC and MAS prioritise knowledge preservation, progressive models promote structural flexibility and sustained adaptation.
- **Emerging trends (2024–2025):** Recent research such as that by Hachohen et al. [8] evidences *LIFO* (Last In, First Out) patterns in continual learning, where examples learned more quickly tend to be better retained, whereas more difficult ones are forgotten more easily. Meanwhile, Sarkar et al. [22] propose *EVCL+* (Elastic Variational Continual Learning Plus), a methodology that combines Fisher regularisation with variational learning to protect the most significant parameters. These trends reflect a growing interest in adaptive optimisation of the weight space, aligning with Bullinaria’s philosophy of adjusting weights through controlled duplication rather than relying on external memories.

- **Recent taxonomies and comparisons:** De Lange et al. [4] establish a structured classification of continual learning methods into three major families: Replay, Regularisation-based, and Parameter Isolation. Their results show that models such as *PackNet* and *iCaRL* achieve greater knowledge retention, while *Memory Aware Synapses (MAS)* exhibits higher robustness to hyperparameter variation. Nevertheless, most of these approaches require explicit memory management or architectural expansion, which increases complexity. Therefore, it is advisable to explore alternatives of internal weight consolidation such as those proposed by Bullinaria, which balance efficiency and stability with minimal computational cost.

9.3.2 Conclusion of the State of the Art

The analysis of the state of the art reveals significant progress in the development of incremental learning models applied to various domains, from robotics and computer vision to cybersecurity and natural language processing. This progress has enabled artificial intelligence systems to gradually adapt to new environments without requiring complete retraining, demonstrating that the ability to learn continuously is essential to achieve truly intelligent and autonomous behaviours.

However, despite the achievements, the reviewed literature shows that incremental learning still faces fundamental challenges related to stability, plasticity, and computational efficiency. In particular, current models tend to fall into one of the two extremes of the stability–plasticity dilemma: either they preserve too much prior knowledge, hindering the incorporation of new information, or they adapt rapidly to recent data at the cost of forgetting what has already been learned. This balance remains one of the main barriers to the consolidation of truly continual learning.

Contemporary strategies to mitigate *catastrophic forgetting*—such as regularisation methods, *replay*, and parameter isolation—have shown partial effectiveness, but present limitations that affect their scalability and practical applicability. Regularisation methods (such as EWC or MAS) require parameter-importance estimations, which entail high computational cost; *replay* methods depend on storing or generating old data, affecting privacy and performance; and parameter isolation methods (such as *PackNet* or *Progressive Networks*) tend to increase network size indefinitely, compromising efficiency in long-term learning contexts.

These limitations highlight the need to explore alternatives that integrate the benefits of these strategies without increasing model complexity or relying on explicit memories. In this context, biologically inspired approaches gain relevance by offering mechanisms that emulate memory consolidation in the human brain—particularly the interaction between the hippocampus (short-term memory) and the neocortex (long-term memory)—thus enabling gradual and efficient knowledge retention.

Within this line, the model proposed by [2] constitutes a key contribution by introducing a dual-memory system based on synaptic weight duplication. This design allows one part of the network to process recent information dynamically (short-term memory), while another part consolidates stable knowledge (long-term memory). This approach offers a viable alternative to architectures that rely on external memories or structural expansions, providing a simpler, more efficient solution with solid neurocomputational foundations.

Nevertheless, the reviewed studies also indicate that Bullinaria’s model, although conceptually robust, has been little explored and lacks extensions that analyse its behaviour with different memory configurations or variations in weight duplication. Most research has been limited to replicating the original implementation without assessing its potential to optimise stability–plasticity in more complex or prolonged scenarios of incremental learning.

Therefore, this research is proposed as a direct extension of that work, with the aim of evaluating and expanding the dual-memory model towards an architecture with multiple weight duplications, analysing its impact on knowledge retention, forgetting rate, and training efficiency. Through this proposal, the objective is to determine whether increasing the number of intermediate memories can favour a more gradual transition between acquisition and consolidation of learning, thereby reducing catastrophic forgetting without significantly increasing computational cost.

In summary, while modern strategies focus on architectural complexity or intensive resource use, the proposed approach returns to a fundamental principle: biologically inspired simplicity as the basis for learning stability. If proven effective, this model could represent an intermediate step between traditional approaches and truly adaptive systems, providing a more sustainable, interpretable, and efficient alternative within the field of incremental learning in artificial neural networks.

10 Methodology

This work is divided into three main stages: the recreation of the base model, the implementation of an extension to the model, and the comparison of the results obtained. Each of these stages is described in detail below:

1. Recreation of the base code

As a first step, the code presented in [2] will be recreated, which describes the implementation of a multilayer neural network using the *backpropagation* training algorithm in the Python programming language. This code will serve as the basis for the following stages.

In addition, the *Optical Digits* dataset will be used and subjected to a preprocessing procedure to remove invalid records and ensure the quality of the data used in training and validating the model.

2. Extension of the base model

Subsequently, an extension to the base model will be developed, with the aim of experimenting with more complex configurations of the neural network. Specifically, more than two layers of duplicated weights will be added, which is expected to improve information retention when employing incremental learning. For this purpose, experiments will be conducted by gradually increasing the number of layers of duplicated weights, in order to analyse the impact of this configuration on the network's learning capacity.

3. Comparison and analysis of results

Once the simulation results have been obtained, a comparison will be carried out between the performance of the base model and the extended model. This analysis will include key metrics such as model accuracy, the rate of information forgetting, and performance in incremental learning.

The results will make it possible to assess whether the modifications introduced into the neural network architecture significantly improve its performance compared with the original implementation.

11 Chapter Organisation

In Chapter 2, the concept and functioning of artificial neural networks will be explained, as well as the activation function, which is a method used by these networks. The types of neural networks will be described, including both multilayer perceptrons and convolutional networks, together with the backpropagation algorithm. The Optdigit dataset, which will be used for training and testing the algorithms, will also be presented. Finally, incremental learning and its algorithm will be described.

In Chapter 3, the algorithm proposed by [2] will be implemented. Subsequently, its performance will be verified, and the results obtained using the same data mentioned in the article will be analysed, in order to confirm that the algorithm's behaviour matches that described by the author.

In Chapter 4, the process of extending the base algorithm to allow the use of more than two layers of duplicated weights will be explained, and the same training and testing data as in the base algorithm will be applied.

Finally, in Chapter 5, using the results obtained, a comparison between both works will be presented to determine whether there was a significant reduction in learning rates. Chapter 6 will then present the conclusions and future work.

12 Gantt Diagram

The following Gantt chart illustrates the planned timeline of the activities comprising the development of this research. Each phase is organised sequentially and bi-monthly, from the formulation of the problem to the professional examination defence. This representation allows visualising the distribution of tasks, execution periods, and the relationship among the various project stages, ensuring systematic and well-structured monitoring of the overall progress.

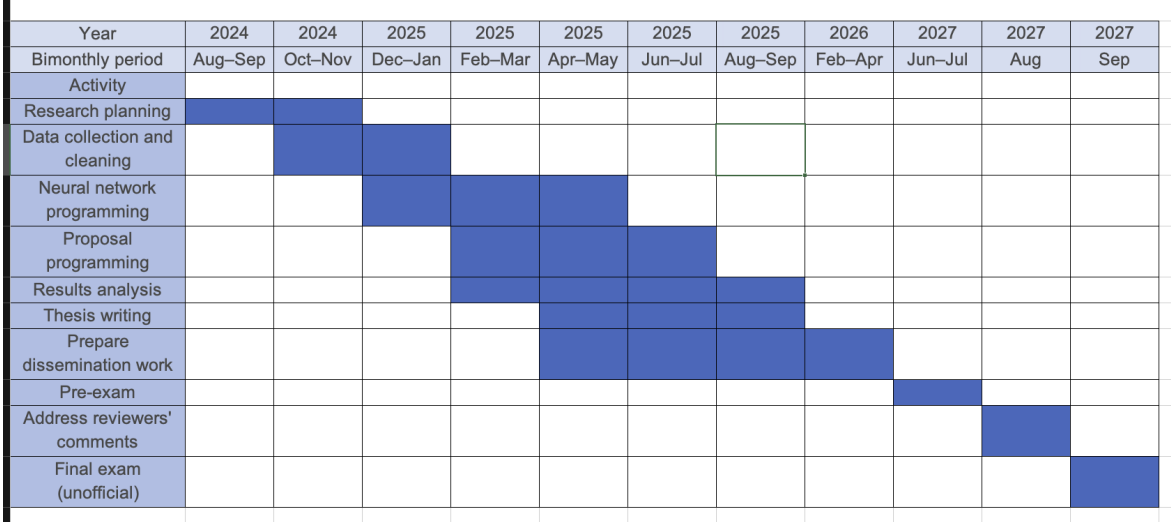


Figure 14: Gantt Chart

References

- [1] RR Ade and PR Deshmukh. Methods for incremental learning: a survey. *International Journal of Data Mining & Knowledge Management Process*, 3(4):119, 2013.
- [2] John A Bullinaria. Evolved dual weight neural architectures to facilitate incremental learning. In *IJCCI*, pages 427–434, 2009.
- [3] Francesco Cerasuolo, Alfredo Nascita, Giampaolo Bovenzi, Giuseppe Aceto, Domenico Ciuonzo, Antonio Pescapè, and Dario Rossi. Memento: A novel approach for class incremental learning of encrypted traffic. *Computer Networks*, 245:110374, 2024.
- [4] Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3366–3385, 2021.
- [5] Ryan Elwell and Robi Polikar. Incremental learning of concept drift in nonstationary environments. *IEEE Transactions on Neural Networks*, 22(10):1517–1531, 2011.
- [6] Douglas H Fisher. Knowledge acquisition via incremental conceptual clustering. *Machine learning*, 2(2):139–172, 1987.
- [7] Christophe G. Giraud-Carrier. A note on the utility of incremental learning. *AI Commun.*, 13:215–224, 2000.
- [8] Guy Hacohen and Tinne Tuytelaars. Forgetting order of continual learning: What is learned first is forgotten last. <https://arxiv.org/abs/xxxx.xxxxx>, 2023. arXiv preprint.
- [9] H. Vega Huerta, A. Cortez Vásquez, Ana María Huayna, Luis Alarcón Loayza, and P. Romero Naupari. Reconocimiento de patrones mediante redes neuronales artificiales. *Revista de investigación de Sistemas e Informática*, 6(2):17–26, 2009.
- [10] Rodrigo Jacznik, Mariano Tassara, Iván D’Uva, Guillermo Baldino, Roxana Silvia Giandini, and Leopoldo Nahuel. Integrando modelo de aprendizaje supervisado al análisis del desempeño de alumnos en cursos virtuales sobre plataformas moodle. In *XXII Congreso Argentino de Ciencias de la Computación (CACIC 2016)*, 2016.
- [11] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- [12] Ai-Jun Li. An improved algorithm for incremental learning learn++. In *2008 International Conference on Wavelet Analysis and Pattern Recognition*, volume 1, pages 310–315. IEEE, 2008.

- [13] Vittorio Lippi, Cristian Camardella, Alessandro Filippeschi, and Francesco Porcini. Identification of gait phases with neural networks for smooth transparent control of a lower limb exoskeleton, 2021.
- [14] Yuan Liu. Incremental learning in deep neural networks, 2015. Unpublished manuscript.
- [15] Luis Miralles Pechuán, Dafne A. Rosso-Pelayo, and Jorge Brieva. Reconocimiento de dígitos escritos a mano mediante métodos de tratamiento de imagen y modelos de clasificación. *Res. Comput. Sci.*, 93:83–94, 2015.
- [16] Darwin Mercado Polo, Luis Pedraza Caballero, and Edinson Martínez Gómez. Comparación de redes neuronales aplicadas a la predicción de series de tiempo. *Prospectiva*, 13(2):88–95, 2015.
- [17] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H. Lampert. icarl: Incremental classifier and representation learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2001–2010, 2017.
- [18] V Renganathan. Overview of artificial neural network models in the biomedical domain. *Bratislava Medical Journal/Bratislavské Lekárske Listy*, 120(7), 2019.
- [19] Raúl Rojas. *The Backpropagation Algorithm*, pages 149–182. Springer Berlin Heidelberg, Berlin, Heidelberg, 1996.
- [20] Paloma Royo. Qué son las redes neuronales y cuál es su aplicación en el marketing, 2021.
- [21] Luis A Cruz Salazar, David J Muñoz Aldana, and Juan A Contreras Montes. Implementación de redes neuronales y lógica difusa para la clasificación de patrones obtenidos por un sónar. In *2013 II International Congress of Engineering Mechatronics and Automation (CIIMA)*, pages 1–6. IEEE, 2013.
- [22] Krisanu Sarkar. Adaptive variance-penalized continual learning with fisher regularization. *arXiv preprint arXiv:2508.16632*, 2025.
- [23] Xing Wei, Shaofan Liu, Yaoci Xiang, Zhangling Duan, Chong Zhao, and Yang Lu. Incremental learning based multi-domain adaptation for object detection. *Knowledge-Based Systems*, 210:106420, 2020.
- [24] Beibei Zhao, Tairan Wu, Fang Fang, Lin Wang, Wenzhang Ren, Xu Yang, Zhangjing Ruan, and Xuejin Kou. Prediction method of 5g high-load cellular based on bp neural network. In *2022 8th International Conference on Mechatronics and Robotics Engineering (ICMRE)*, pages 148–151. IEEE, 2022.

A Phases of the Backpropagation Process

A.1 Forward Propagation

In this phase, the input values are introduced into the network, the output is computed, and no weight adjustment occurs, as only the activation value is calculated.

This is achieved through matrix multiplication, where each neuron receives the output of the neurons from the previous layer multiplied by their respective weights, according to the following equation:

$$z_j = \sum_i (w_{ji} \cdot x_i) + b_j$$

Where:

- w_{ji} = weight of the connection between neuron i in the previous layer and neuron j in the current layer.
- x_i = output of neuron i from the previous layer.
- b_j = bias of neuron j .

A.2 Backward Propagation

In this case, the network weights are adjusted by propagating the error backward (from the output towards the hidden layers) using gradient descent. To achieve this, the error must first be calculated, and the loss is computed as follows: $L = \frac{1}{2}(y - \hat{y})^2$. With these values, backpropagation is performed using the chain rule of differential calculus. This process is divided into two parts: the calculation for the output layer and for the hidden layers.

- **Output Layer j :**

$$\frac{\partial L}{\partial z_j} = \frac{\partial L}{\partial \hat{y}_j} \cdot \frac{\partial \hat{y}_j}{\partial z_j}$$

Where:

- $\frac{\partial L}{\partial \hat{y}_j} = (\hat{y}_j - y_j)$ (derivative of the loss).
- $\frac{\partial \hat{y}_j}{\partial z_j}$ depends on the activation function. For example, if it is sigmoid:

$$\sigma'(z_j) = \sigma(z_j)(1 - \sigma(z_j))$$

- **Hidden Layers:**

$$\frac{\partial L}{\partial z_j} = \sum_k \left(\frac{\partial L}{\partial z_k} \cdot w_{kj} \cdot \sigma'(z_k) \right)$$

Here, w_{kj} represents the weights of the next layer.

The weights (w_{ji}) and biases (b_j) can then be updated using the gradient and a learning rate (η):

$$w_{ji} \leftarrow w_{ji} - \eta \cdot \frac{\partial L}{\partial w_{ji}}$$

$$b_j \leftarrow b_j - \eta \cdot \frac{\partial L}{\partial b_j}$$

Where:

$$\frac{\partial L}{\partial w_{ji}} = \frac{\partial L}{\partial z_j} \cdot x_i \quad \text{and} \quad \frac{\partial L}{\partial b_j} = \frac{\partial L}{\partial z_j}$$